2026-10-25 09:30:43 You

###  Level 10 — Production / DevOps Integration

These are useful when connecting Ansible with your other DevOps knowledge.

101. **Ansible + AWS**
102. **Dynamic AWS inventory**
103. **Ansible + Terraform**
104. **Ansible + Jenkins**
105. **Ansible in CI/CD**
106. **Ansible + Docker**
107. **Ansible + Kubernetes**
108. **Configuration drift**
109. **Rolling deployment**
110. **Zero/minimal-downtime deployment**
111. **Environment management — dev/stage/prod**
112. **Ansible project structure**
113. **Logging & troubleshooting**
114. **Ansible linting**
115. **Testing Ansible**

ChatGPT gpt-6

## Ansible Level 10 — Production / DevOps Integration

**Goal:** Understand how Ansible works with AWS, Terraform, Jenkins, Docker, Kubernetes, and real production deployment workflows.

The key idea: **Terraform creates infrastructure, Ansible configures servers, and Jenkins automates the delivery process.**

### 101. Ansible + AWS

**Purpose:** Configure and manage AWS resources or servers running on AWS.

Common use cases:

- Install packages on EC2 instances.
- Configure Nginx, Java, and application services.

- Deploy application files.
- Manage AWS resources using AWS-specific Ansible modules.

Example: configure an EC2 instance after it is created.

```yaml
</> YAML

- name: Configure EC2 web server
  hosts: web
  become: true
  tasks:
    - name: Install Nginx
      ansible.builtin.apt:
        name: nginx
        state: present
        update_cache: true

    - name: Start Nginx
      ansible.builtin.service:
        name: nginx
        state: started
        enabled: true
```

**Production flow:**

1. EC2 instance is running.
2. Ansible connects through SSH.
3. Ansible installs and configures the required software.
4. The server becomes ready to host the application.

**Remember:** AWS provides the infrastructure; Ansible configures it.

### 102. Dynamic AWS Inventory

**Purpose:** Automatically discover EC2 instances instead of manually maintaining their IP addresses in an inventory file.

Static inventory:

```ini
</> INI

[web]
10.0.1.10
10.0.1.11
```

Dynamic inventory discovers instances from AWS using tags, regions, and other filters.

Example inventory configuration, `inventory.aws_ec2.yml`:

```yaml
plugin: amazon.aws.aws_ec2
regions:
  - us-east-1

filters:
  instance-state-name: running

keyed_groups:
  - key: tags.Role
    prefix: role
```

Install the required collection:

```bash
ansible-galaxy collection install amazon.aws
```

Check discovered hosts:

```bash
ansible-inventory -i inventory.aws_ec2.yml --graph
```

**Production use:** Tag EC2 instances with `Role=web` or `Role=db` and organize deployments around those tags.

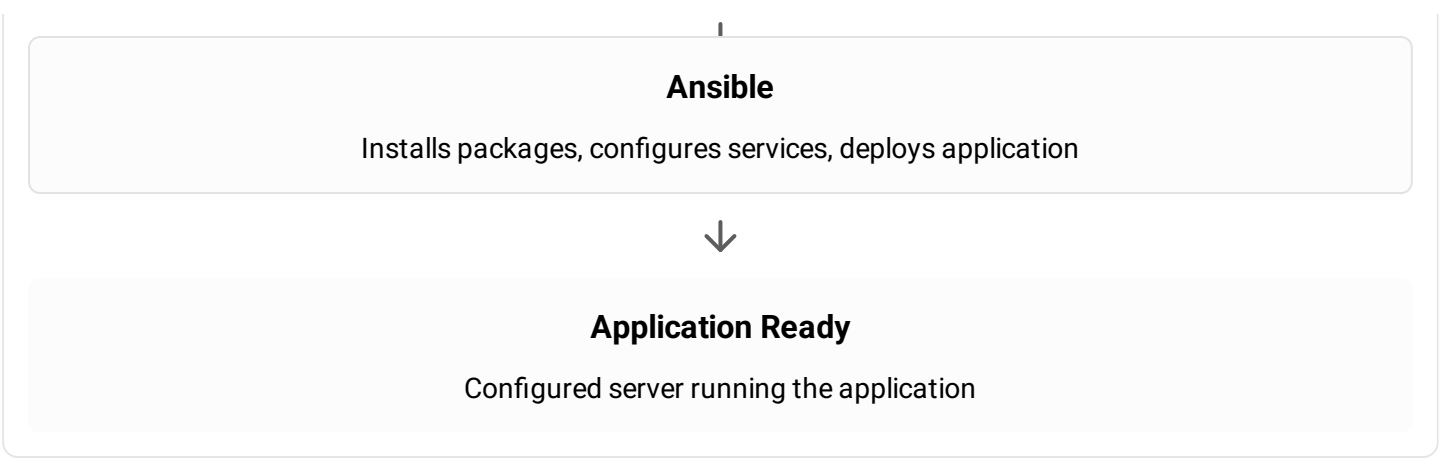
**Remember:** Dynamic inventory discovers hosts; it does not create EC2 instances.

### 103. Ansible + Terraform

**Purpose:** Use Terraform to create infrastructure and Ansible to configure it.

**Terraform**

Creates VPC, subnets, security groups, EC2 instances




Example Terraform output:

```hcl
output "web_public_ip" {
  value = aws_instance.web.public_ip
}
```

Get the IP:

```bash
terraform output -raw web_public_ip
```

You can use this output to populate Ansible inventory or feed it into an automated deployment workflow.

**Production use:** Terraform manages infrastructure lifecycle; Ansible manages operating-system and application configuration.

**Important:** Avoid blindly running Terraform and Ansible together without checking dependencies, SSH readiness, and infrastructure outputs.

### 104. Ansible + Jenkins

**Purpose:** Run Ansible playbooks automatically from Jenkins jobs or pipelines.

Example Jenkins pipeline:

```groovy
pipeline {
    agent any
```

```
    stages {
        stage('Checkout') {
            steps {
                git 'https://github.com/example/app-config.git'
            }
        }

        stage('Deploy') {
            steps {
                sh 'ansible-playbook -i inventory/hosts.ini deploy.yml'
            }
        }
    }
}
```

For this to work, the Jenkins execution environment needs Ansible, inventory, network access, and authorized credentials.

**Production considerations:**

- Store secrets in Jenkins Credentials, not directly in the Jenkinsfile.
- Use an SSH key or another approved authentication method.
- Restrict which jobs and users can deploy to production.
- Use a controlled production approval step when appropriate.

**Remember:** Jenkins triggers and coordinates the pipeline; Ansible performs the configuration and deployment tasks.

## 105. Ansible in CI/CD

**Purpose:** Make configuration and deployment repeatable instead of manually running commands on servers.

Typical pipeline:

**1. Developer**

Pushes code to Git

↓

**2. Jenkins**

Runs build and tests

↓

**3. Artifact**

Creates a JAR or Docker image

↓

**4. Ansible**

Deploys the version to target servers

↓

**5. Verification**

Checks application health

Example deployment task:

```yaml
- name: Copy application JAR
  ansible.builtin.copy:
    src: target/app.jar
    dest: /opt/app/app.jar
    owner: appuser
    group: appuser
    mode: "0644"
```

**Remember:** CI validates and builds; CD delivers and deploys. Ansible is commonly used for the deployment/configuration part, though the exact division varies by pipeline.

### 106. Ansible + Docker

**Purpose:** Install Docker, configure Docker hosts, and manage containers.

Common tasks:

- Install Docker Engine.
- Log in to a registry securely.
- Pull an image.
- Start or update containers.
- Configure container environment variables and ports.

Example using the Docker collection:

```yaml
- name: Run application container
  hosts: docker_hosts
```

```yaml
  become: true
  tasks:
    - name: Start application
      community.docker.docker_container:
        name: myapp
        image: "myrepo/myapp:1.2.0"
        state: started
        restart_policy: always
        published_ports:
          - "8080:8080"
```

Install the collection:

```bash
ansible-galaxy collection install community.docker
```

**Remember:** Docker runs the container; Ansible automates its configuration and lifecycle.

This example assumes Docker is already installed and accessible on the target host.

### 107. Ansible + Kubernetes

**Purpose:** Automate tasks around Kubernetes environments.

Ansible can:

- Install and configure Kubernetes-related software.
- Prepare machines to become cluster nodes.
- Deploy Kubernetes resources using modules or `kubectl`.
- Manage supporting infrastructure and application configuration.

Example: deploy an application to an existing cluster.

```yaml
- name: Deploy application to Kubernetes
  hosts: localhost
  connection: local
  tasks:
    - name: Apply Kubernetes manifest
      kubernetes.core.k8s:
        state: present
        src: deployment.yml
```

Install the collection:

```
</> Bash

ansible-galaxy collection install kubernetes.core
```

This example requires the Kubernetes Python client and valid cluster credentials to be configured.

**Production distinction:**

- **Ansible:** Useful for infrastructure preparation, configuration, and deployment automation.
- **Kubernetes:** Schedules containers, manages replicas, and maintains the desired state of workloads.
- **Helm:** Packages Kubernetes applications into reusable charts.

**Remember:** Ansible can deploy resources to Kubernetes, but Kubernetes itself manages the running workloads.

### 108. Configuration Drift

**Purpose:** Detect or correct differences between the intended configuration and the actual server configuration.

Example:

Your Ansible playbook specifies that Nginx must be installed. Someone manually removes it from a production server.

The server no longer matches the desired configuration. This is **configuration drift**.

Example:

```
</> YAML

- name: Ensure Nginx is installed
  ansible.builtin.apt:
    name: nginx
    state: present
```

Run the playbook again:

```
</> Bash
```

```
ansible-playbook -i inventory/hosts.ini site.yml
```

Ansible reinstalls Nginx if it is missing.

To preview potential changes:

```bash
ansible-playbook -i inventory/hosts.ini site.yml --check --diff
```

**Important:** Check mode is a simulation, not a guarantee. Some tasks and modules cannot accurately predict all changes in check mode.

**Production use:** Run scheduled configuration checks and investigate unexpected changes instead of automatically overwriting every change without review.

**Remember:** Idempotency helps Ansible converge a system toward the desired configuration.

### 109. Rolling Deployment

**Purpose:** Update application servers in batches rather than updating every server simultaneously.

Suppose you have four web servers. You want to update one server at a time while the others continue serving requests.

Use `serial`:

```yaml
- name: Rolling deployment
  hosts: web
  become: true
  serial: 1

  tasks:
    - name: Deploy application
      ansible.builtin.copy:
        src: app.jar
        dest: /opt/app/app.jar

    - name: Restart application
      ansible.builtin.service:
        name: myapp
        state: restarted
```

If the inventory contains four hosts, Ansible processes one host per batch.

**Production requirement:** Configure load-balancer draining and health checks where necessary. `serial: 1` alone does not remove a server from traffic before restarting it.

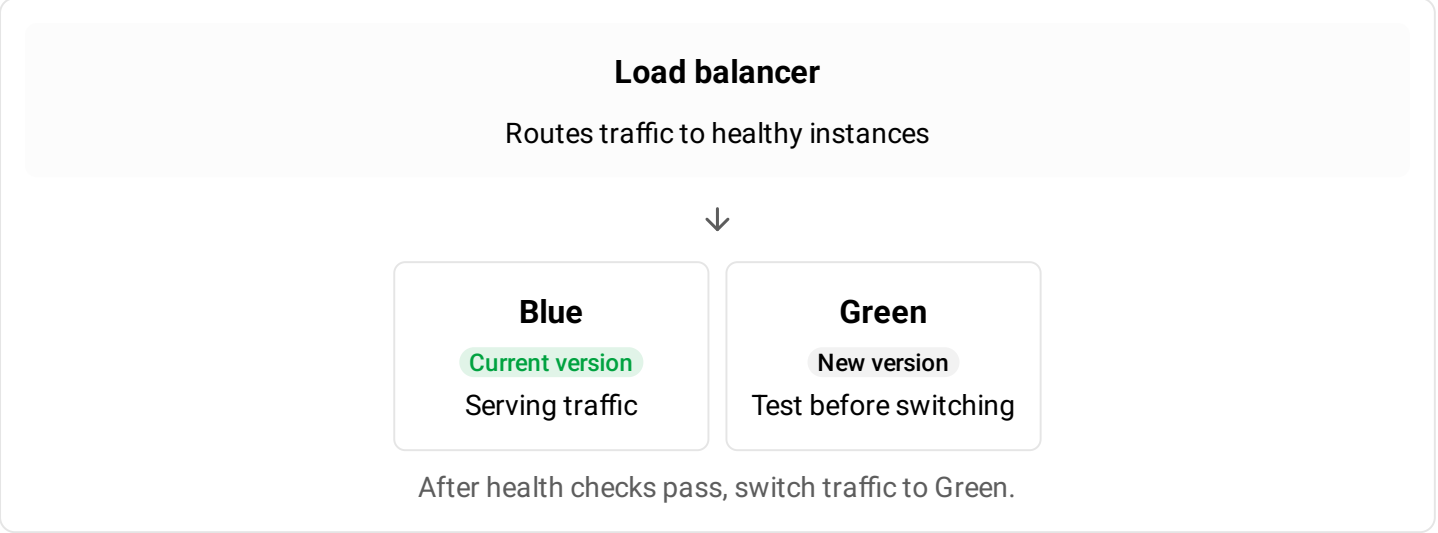
**Remember:**

- `serial: 1` — one host per batch.
- `serial: 2` — two hosts per batch.
- `serial: "25%"` — a percentage of the host group per batch, subject to Ansible's batch-sizing rules.

### 110. Zero/Minimal-Downtime Deployment

**Purpose:** Release a new application version while minimizing or avoiding user-visible downtime.

A normal restart can briefly interrupt service. A safer deployment can use two application versions or environments.




This is the **blue-green deployment** pattern.

Typical steps:

1. Keep the current version running.
2. Deploy the new version to a separate environment.
3. Run health checks and smoke tests.
4. Switch traffic to the new version.
5. Keep a rollback path to the previous version.

Ansible can automate deployment and verification steps, while a load balancer or deployment platform handles traffic switching.

**Important:** Zero downtime is not guaranteed. Database migrations, session handling, traffic switching, and backward compatibility must also be considered.

**Remember:** Rolling deployment updates instances in batches; blue-green deployment switches traffic between two environments.

### 111. Environment Management — Dev / Stage / Prod

**Purpose:** Use the same automation while applying different settings to different environments.

Example structure:

```
</> Plain text

ansible-project/
├──── inventory/
│     ├──── dev/
│     │     └──── hosts.ini
│     ├──── stage/
│     │     └──── hosts.ini
│     └──── prod/
│           └──── hosts.ini
├──── group_vars/
│     └──── all.yml
├──── playbooks/
│     └──── deploy.yml
└──── ansible.cfg
```

Example `inventory/dev/hosts.ini`:

```
</> INI

[web]
dev-web-01 ansible_host=10.0.1.10
```

Example `inventory/prod/hosts.ini`:

```
</> INI

[web]
prod-web-01 ansible_host=10.0.2.10
```

Run the same playbook against different inventories:

```
</> Bash
```

```
ansible-playbook -i inventory/dev/hosts.ini playbooks/deploy.yml
```

```
</> Bash

ansible-playbook -i inventory/prod/hosts.ini playbooks/deploy.yml
```

You can also maintain environment-specific variables, such as application versions, ports, and feature flags.

**Production rules:**

- Keep production credentials separate and protected.
- Use approvals and access restrictions for production deployment.
- Promote a tested artifact from dev to stage to prod rather than rebuilding a different artifact for each environment.

**Remember:** Same playbook, different inventory and configuration.

## 112. Ansible Project Structure

**Purpose:** Organize automation so it is reusable, maintainable, and easier for teams to understand.

Example structure:

```
</> Plain text

ansible-project/
├── ansible.cfg
├── requirements.yml
├── inventory/
│   ├── dev/
│   └── prod/
├── playbooks/
│   ├── site.yml
│   └── deploy.yml
├── roles/
│   ├── common/
│   │   ├── tasks/
│   │   │   └── main.yml
│   │   ├── handlers/
│   │   │   └── main.yml
│   │   ├── templates/
│   │   ├── files/
│   │   ├── defaults/
```

```
│   │   └── vars/
│   └── nginx/
│       └── tasks/
│           └── main.yml
└── group_vars/
    └── all.yml
```

What the important files do:

| File/folder | Purpose |
|---|---|
| `ansible.cfg` | Project-level Ansible settings |
| `inventory/` | Defines or discovers target hosts |
| `playbooks/` | Defines automation workflows |
| `roles/` | Contains reusable automation components |
| `group_vars/` | Variables shared by host groups |
| `requirements.yml` | Lists external roles or collections to install |

**Production benefit:** Teams can reuse the same role for multiple applications and environments without copying the same tasks everywhere.

### 113. Logging & Troubleshooting

**Purpose:** Understand what happened during a playbook execution and identify failures.

Useful commands:

```bash
# Test connectivity
ansible web -i inventory/hosts.ini -m ansible.builtin.ping

# Run with more detail
ansible-playbook -i inventory/hosts.ini site.yml -v

# Increase diagnostic detail
ansible-playbook -i inventory/hosts.ini site.yml -vvv

# Check syntax
ansible-playbook site.yml --syntax-check

# List matching hosts
ansible-playbook -i inventory/hosts.ini site.yml --list-hosts
```

Verbosity levels:

| Option | Use |
| --- | --- |
| `-v` | More execution details |
| `-vv` | Additional task information |
| `-vvv` | Detailed troubleshooting, often useful for connection issues |
| `-vvvv` | Very detailed connection/debug information |

You can also configure a log file in `ansible.cfg`:

```ini
[defaults]
log_path = ./ansible.log
```

**Production troubleshooting order:**

1. Confirm inventory contains the intended hosts.
2. Verify SSH access, credentials, and privilege escalation.
3. Check the failing task and its error message.
4. Verify variables, file paths, permissions, and dependencies.
5. Reproduce safely, then fix and rerun.

**Security:** Debug output and logs can expose secrets. Protect log files and use `no_log: true` for tasks handling sensitive data.

## 114. Ansible Linting

**Purpose:** Find common mistakes and inconsistent practices in Ansible playbooks and roles before execution.

Install:

```bash
pip install ansible-lint
```

Run against a playbook:

```bash
ansible-lint playbooks/site.yml
```

Example of a task:

```yaml
- name: Install Nginx
  ansible.builtin.apt:
    name: nginx
    state: present
```

Linting can flag issues such as:

- Missing or unclear task names.
- Deprecated syntax or problematic patterns.
- Incorrect formatting or risky practices.
- Inconsistent module usage.

**Production use:** Add `ansible-lint` to a pull-request or CI pipeline so automation is reviewed before being merged.

**Remember:** Linting checks code quality; it does not prove the playbook will work correctly on the target servers.

## 115. Testing Ansible

**Purpose:** Verify that automation is valid and behaves as expected before making production changes.

A practical testing sequence:

**1. Syntax check**

Checks playbook syntax.

```
ansible-playbook site.yml --syntax-check
```

**2. Lint**

Checks common quality and style issues.

```
ansible-lint site.yml
```

**3. Check mode**

Simulates supported changes where possible.

```
ansible-playbook -i inventory/hosts.ini site.yml --check
```

**4. Test environment**

Validates actual behavior in a controlled environment.

```
Run against a disposable VM or dev host
```

**5. Production rollout**

Reduces risk and makes failures easier to contain.

```
Deploy in batches and verify health
```

Additional tools:

- **Molecule:** Tests Ansible roles through defined test scenarios.
- **Integration tests:** Verify that configured services work together.
- **Smoke tests:** Check essential functionality after deployment.

Example health check:

```yaml
- name: Verify application responds
  ansible.builtin.uri:
    url: http://localhost:8080/health
    status_code: 200
  register: health_result
  retries: 5
  delay: 3
  until: health_result.status == 200
```

This assumes the application exposes `/health` and returns HTTP 200 when healthy.

**Remember:** Syntax checking, linting, and check mode are useful safeguards, but none replaces testing against a real test environment.

---

## Level 10 — Quick revision

| Concept | Remember this |
|---|---|
| Ansible + AWS | Configure AWS-hosted servers and manage supported AWS resources |
| Dynamic inventory | Discover hosts automatically |
| Ansible + Terraform | Terraform creates infrastructure; Ansible configures it |
| Ansible + Jenkins | Jenkins triggers playbooks in a pipeline |

| | |
|---|---|
| Ansible in CI/CD | Automate repeatable build/deployment workflows |
| Ansible + Docker | Automate container-host and container configuration |
| Ansible + Kubernetes | Automate cluster preparation and Kubernetes resource operations |
| Configuration drift | Actual configuration differs from the desired configuration |
| Rolling deployment | Update servers in batches |
| Blue-green deployment | Switch traffic between old and new environments |
| Environment management | Reuse playbooks with environment-specific inventories and variables |
| Project structure | Organize inventories, roles, playbooks, and variables |
| Troubleshooting | Inspect inventory, connectivity, errors, and logs |
| Ansible lint | Detect common code-quality issues |
| Testing | Validate syntax, behavior, and application health |

## What should you prioritize?

Given your DevOps interview and production-preparation goals, focus in this order:

1. **Must understand deeply:** Ansible + Terraform, Ansible + Jenkins, dynamic inventory, environments, configuration drift, rolling deployment, troubleshooting.
2. **Know the workflow:** Docker, Kubernetes, linting, testing, and CI/CD integration.
3. **Learn hands-on next:** Build one small project rather than separate labs for every concept.

### One practical project to connect Level 1–10

Build a small deployment pipeline:

1. Terraform creates an EC2 instance.
2. Ansible dynamic inventory discovers the instance.
3. Jenkins runs the deployment playbook.
4. Ansible installs Java or Nginx and deploys the application.
5. A health check verifies the deployment.
6. A rolling strategy updates multiple instances in batches.
7. Ansible lint and syntax checks run before deployment.

This one project covers most of the important Ansible concepts in a realistic DevOps workflow.